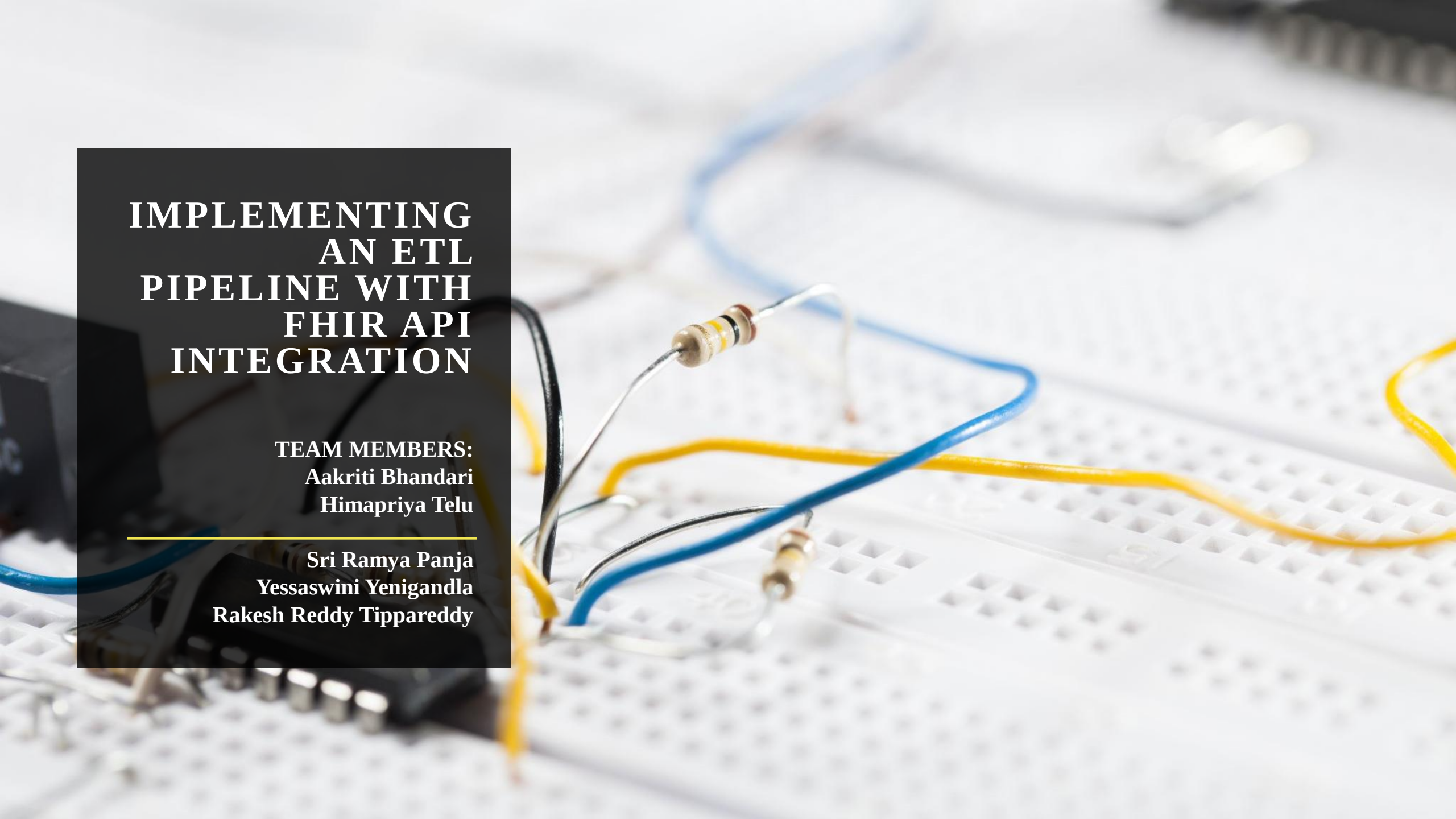




IMPLEMENTING AN ETL PIPELINE WITH FHIR API INTEGRATION

TEAM MEMBERS:

Aakriti Bhandari

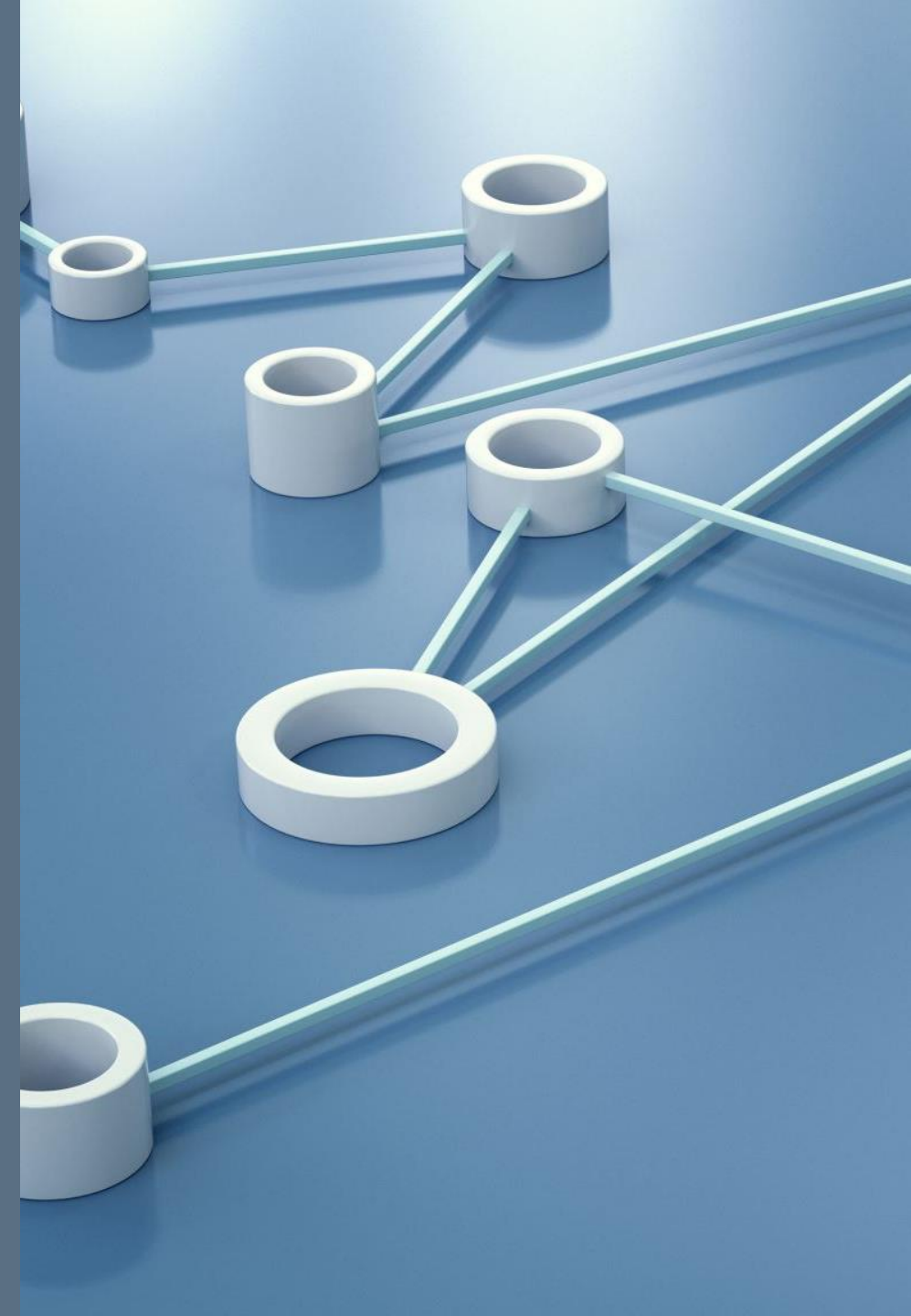
Himapriya Telu

Sri Ramya Panja
Yessaswini Yenigandla
Rakesh Reddy Tippareddy

INTRODUCTION

Overview of FHIR APIs in Healthcare Data Exchange

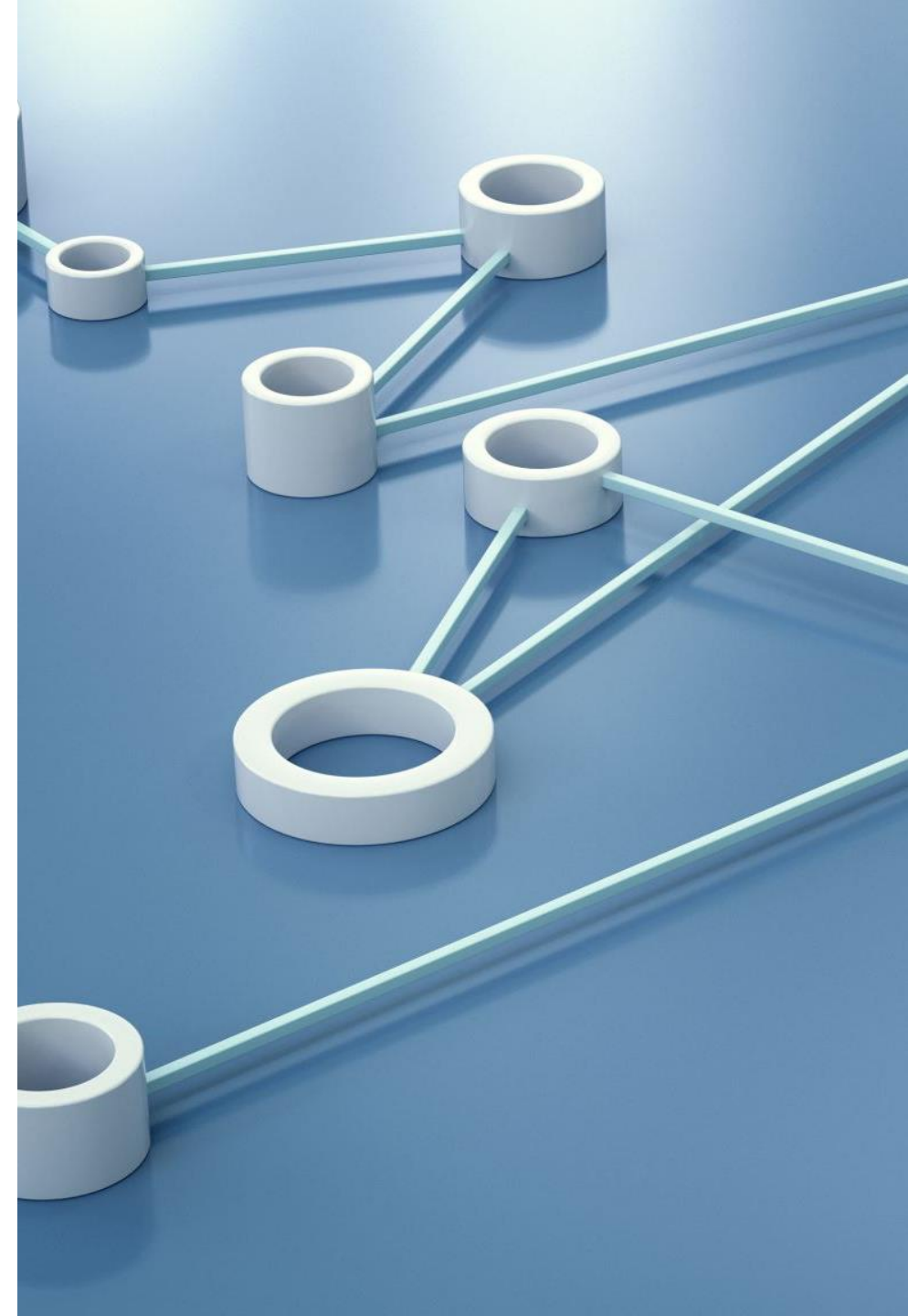
- FHIR (Fast Healthcare Interoperability Resources) enables standardized healthcare data exchange.
- Key Features:
 - Built on RESTful APIs and uses JSON for data formatting.
 - Utilizes modular resources like Patient, Observation, and Procedure.
 - Supports real-time data sharing across systems.
- Benefits:
 - Facilitates app development (e.g., SMART on FHIR applications).
 - Promotes interoperability and consistency in patient care.
 - Enables scalable solutions with comprehensive health record access.



INTRODUCTION

High-Level Explanation of the ETL Process

- ETL Definition: Extract, Transform, Load process used for integrating and preparing data.
- Steps:
 - Extract:
 - Pulls data from various sources (e.g., FHIR APIs, EHRs, labs).
 - Transform:
 - Cleans, standardizes, and formats data (e.g., mapping to FHIR resource types).
 - Load:
 - Stores processed data in a central repository (e.g., data warehouse).
- Purpose:
 - Prepares data for analytics, decision-making, and improved interoperability.



ETL PROCESS



Github repository link for our ETL project:

https://github.iu.edu/himatelu/Group3_Project_Final

The ETL process (Extract, Transform, Load) is a data integration workflow used to collect data from various sources, process it into a standardized, clean format, and load it into a target system.

In the Extract phase, raw data is gathered from databases, APIs, or files.

In Transform, the data is cleaned, standardized, and enriched to ensure consistency and usability.

Finally, in the Load phase, the processed data is stored ready for analysis and reporting, enabling insights and decision-making.

E COMPONENT OF ETL (EXTRACT)

```
def search_patient_by_name(name_string):  # sripanja
    url = f'{BASE_URL}/Patient?name={name_string}'
    response = requests.get(url=url, headers=get_headers())
    print(response.url)
    pprint(response.json())
```

Here, the script uses `requests.get` to connect to Open EMR to retrieve data using patient's name

```
def get_access_token_from_file():  # usage  # sripanja
    file_path = Path(data_dir / "access_token.json")
    if not file_path.exists():
        print("Error: access_token.json file not found.")
        return None
    try:
        with open(file_path, 'r') as json_file:
            json_data = json.load(json_file)
            access_token = json_data.get("access_token")
            return access_token
    except (json.JSONDecodeError, KeyError) as e:
        print(f"Error reading access token from file: {e}")
        return None
```

The script reads an access token from a file (`access_token.json`). If the file is missing or the token is invalid, it handles errors by returning `None`

EXECUTION AND RETRIEVAL OF PATIENT'S CONDITION

```
if __name__ == '__main__':  
    patient_resource_id = '985ac75c-54cd-47ab-afe1-93d52db5ba48'  
    search_patient_by_name(name_string='Bailey')  
    get_fhir_patient(patient_resource_id)  
    search_condition(patient_resource_id)
```

The Python script is executed by running the `__main__` block at the bottom of the script, which calls the `get_fhir_patient` and `search_condition` functions.

The `search_condition` function retrieves conditions associated with a specific patient. It queries the FHIR API for conditions using the `patient_resource_id`. The script then processes these conditions and retrieves relevant SNOMED CT information (like the direct parent and body site) to populate the `new_condition_dict` which is a dictionary

```
def search_condition(patient_resource_id): 1 usage  sripanja +1  
    url = f'{BASE_URL}/Condition?patient={patient_resource_id}'  
    response = requests.get(url=url, headers=get_headers())  
    data = response.json()  
    #pprint(data)  
    if 'entry' in data:  
        conditions = data['entry']  
        thirty_condition = conditions[30]  
        snomed_code_30th_cond = thirty_condition['resource']['code']['coding'][0]['code']  
        parent = get_direct_parent(concept_id=snomed_code_30th_cond)  
        parent_code = parent[0]  
        parent_description = parent[1]  
        finding_site = get_body_site(concept_id=snomed_code_30th_cond)  
        site_code = finding_site[0]  
        site_description = finding_site[1]  
        print(site_code)
```

T COMPONENT OF ETL (TRANSFORM)

Parent Term: Microalbuminuric diabetic nephropathy

- Represents the broader condition of kidney damage due to diabetes with microalbuminuria.

Child Term: Persistent microalbuminuria due to type 2 diabetes mellitus

- A specific condition under the parent term, describing the persistence of microalbuminuria caused by type 2 diabetes.

These parent-child term relationships were used to properly classify and link conditions, ensuring accurate data transformation for integration into the target API.

T COMPONENT OF ETL (TRANSFORM)

Missing fields in the target API were handled by assigning default or placeholder values to ensure data integrity by creating a dictionary for patient and conditions without compromising accuracy. Inconsistent fields, such as format discrepancies (e.g., numerical to string), were adjusted to match the target API's expected structure and terminology.

Handling missing fields

```
new_patient_dict['address'][0]['district']= 'Not found'
```

SNOMED CT Value Adjustments

```
new_condition_dict['code']['coding'][0]['code'] = parent_code
```

Address field mapping using dictionary new_patient_dict

```
new_patient_dict['address'][0]['state'] = address_state  
new_patient_dict['address'][0]['postalCode'] = address_postcode
```

```

try:
    headers = {
        'Accept': "application/json"
    }
    url = PRIMARY_CARE_SERVER_URL + '/' + 'Patient'
    response = requests.post(url=url, json=new_patient_dict, headers=headers)
    if response.status_code == 200 or response.status_code == 201:
        response_data = response.json()
        with open(data_dir / 'patient_resource_id.txt', 'w') as f:
            f.write(response_data['id'])
        print('Patient resource ID created.')
    else:
        print(f'Error - {response.status_code}')
except Exception as e:
    print('Could not execute the process of creating patient resource')

```

```

try:
    headers = {
        'Accept': "application/json"
    }
    url = PRIMARY_CARE_SERVER_URL + '/' + 'Condition'
    response = requests.post(url=url, json=new_condition_dict, headers=headers)
    if response.status_code == 200 or response.status_code == 201:
        response_data = response.json()
        print(response_data)
        print('Condition resource created.')
    else:
        print(f'Error - {response.status_code}')
except Exception as e:
    print('Could not process the request')
else:
    print('No results found')

```

L COMPONENT OF ETL (LOADING)

Task 1 loading the patient details and parent term as conditions into primary care EHR FHIR server

L COMPONENT OF ETL (LOADING)

Task 2 of loading the child term conditions to primary care EHR FHIR server

```
try:
    headers = {
        'Accept': "application/json"
    }
    url = PRIMARY_CARE_SERVER_URL + '/' + 'Condition'
    response = requests.post(url=url, json=new_condition_dict, headers=headers)
    if response.status_code == 200 or response.status_code == 201:
        response_data = response.json()
        #pprint(response_data)
        print('Condition resource created.')
    else:
        print(f'Error - {response.status_code}')
except Exception as e:
    print(e)
```

L COMPONENT OF ETL (LOADING)

Task 3 creating blood pressure
Observation for the patient in
Task 1 of primary care EHR
FHIR server

```
def post_data(file_name, resource_name): 1 usage  sripanja *
    # Define the URL of the API endpoint
    url = BASE_SERVER_URL + '/' + f'{resource_name}'
    data = read_data(name_of_the_file=file_name)
    # Define headers, including content-type as JSON
    headers = {
        "Content-Type": "application/json",
    }

    try:
        # Send POST request
        response = requests.post(url, json=data, headers=headers)
        # Check if the request was successful
        if response.status_code == 200 or response.status_code == 201:
            print(f"Data posted successfully! - {resource_name}")
            print("Response:", response.json())
        else:
            print(f"Failed to post data - {resource_name}. Status code:", response.status_code)
            print("Error:", response.text)
    except requests.exceptions.RequestException as e:
        print("Error during request:", e)
```

L COMPONENT OF ETL (LOADING)

Task 4 for creating a procedure
for the patient created in Task 1
in primary care EHR FHIR server

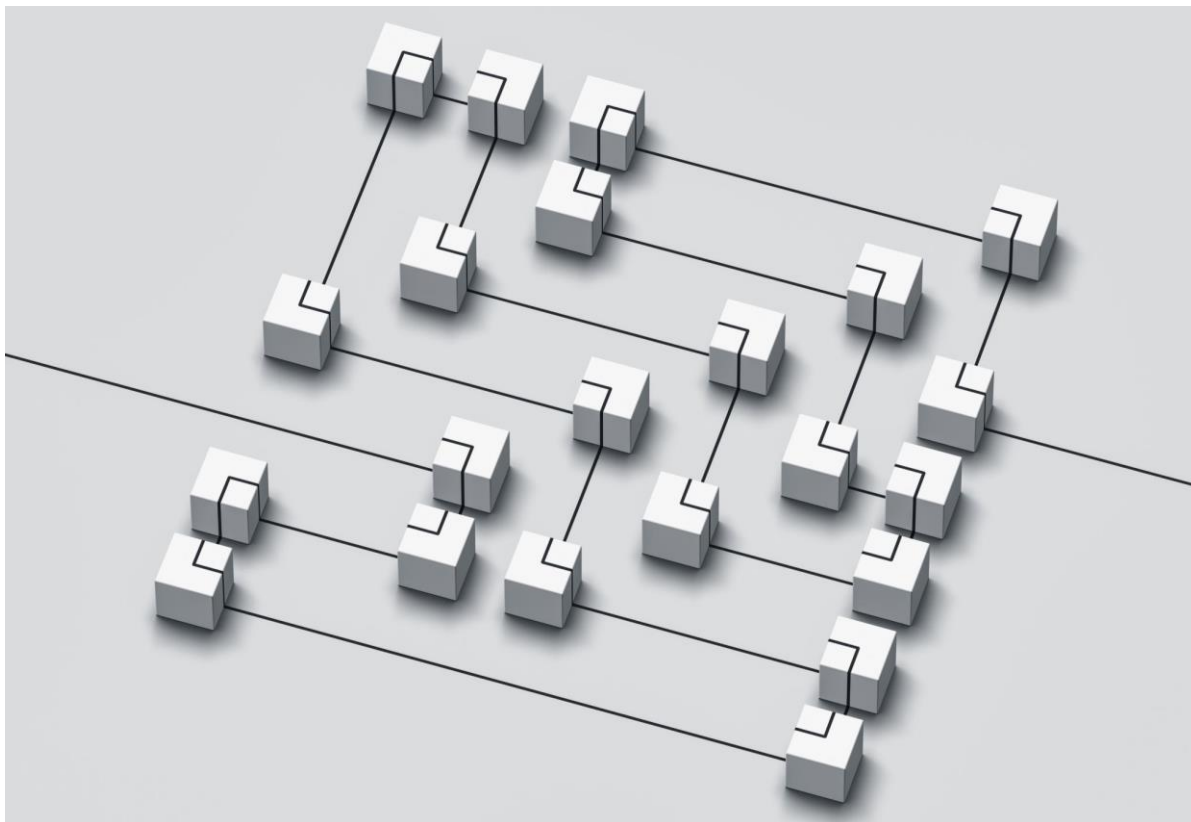
```
def post_data(file_name, resource_name): 1 usage  👤 sripanja +1
    # Define the URL of the API endpoint
    url = BASE_SERVER_URL + '/' + f'{resource_name}'
    data = read_data(name_of_the_file=file_name)
    # Define headers, including content-type as JSON
    headers = {
        "Content-Type": "application/json",
    }
    with open(data_dir / 'patient_resource_id.txt', 'r') as f:
        patient_resource_id = f.read().strip()

    patient_obj['subject']['reference'] = f"Patient/{patient_resource_id}"
    try:
        # Sending POST request
        response = requests.post(url, json=data, headers=headers)
        # Checking if the request was successful
        if response.status_code == 200 or response.status_code == 201:
            print(f>Data posted successfully! - {resource_name}")
            print("Response:", response.json())
        else:
            print(f"Failed to post data - {resource_name}. Status code:", response.status_code)
            print("Error:", response.text)
    except requests.exceptions.RequestException as e:
        print("Error during request:", e)
```

INSIGHTS FROM THE DATA

Data Volume and Clinical Value: The OpenEMR FHIR API contains a significant volume of patient data, offering a rich source of information for clinical analysis and health trend insights.

Enhanced Interoperability: Integrating this data with the Hermes Terminology Server, and utilizing SNOMED CT parent-child mappings, helped structure the data more effectively, improving interoperability and clinical value.



CHALLENGES

API Complexities:

We also encountered an issue when attempting to retrieve the child's terms for a condition. Direct retrieval wasn't possible through the Hermes and SNOMED endpoints.

To address this, we used ECL queries with the <! operator to get the direct children of the term and integrate the results into our solution.

Data Inconsistencies:

We couldn't find the body site and severity keys in the JSON file obtained from the OpenEMR patient resource. These were required to assign values to the dictionary for the primary care EHR, highlighting a data inconsistency.

To resolve this, we assigned values to the primary care EHR dictionary by making a GET request to the Hermes terminology server. This allowed us to retrieve the necessary information directly from SNOMED CT and update the condition resource in the primary care.

KEY LESSON LEARNED

Gained a deeper understanding of the ETL process (Extract, Transform, Load) by implementing it in Python, which enhanced our overall grasp of the project beyond theoretical knowledge.

Improved comprehension of data structures and mapping by working with JSON files, converting them into dictionaries, and resolving errors during mapping between different APIs.

Realized the value of validating work through multiple platforms (e.g., Postman, the Primary Care EHR server, and code outputs), which provided a more comprehensive view of the data and processes.



BEST PRACTICES FOR FHIR APIS

- **Understand FHIR Resources:**

- Gain familiarity with FHIR resources (e.g., Patient, Observation, Procedure) and their interrelationships to design effective data exchanges.

- **Secure Data Exchange:** Use robust authentication methods (e.g., OAuth2), encryption, and ensure compliance with FHIR standards for data validation.

- **Optimize API Interactions:**

- Apply filters and divide large datasets into smaller parts to handle them more easily.
- Reduce the time it takes for API calls to complete, making the system faster and more efficient.



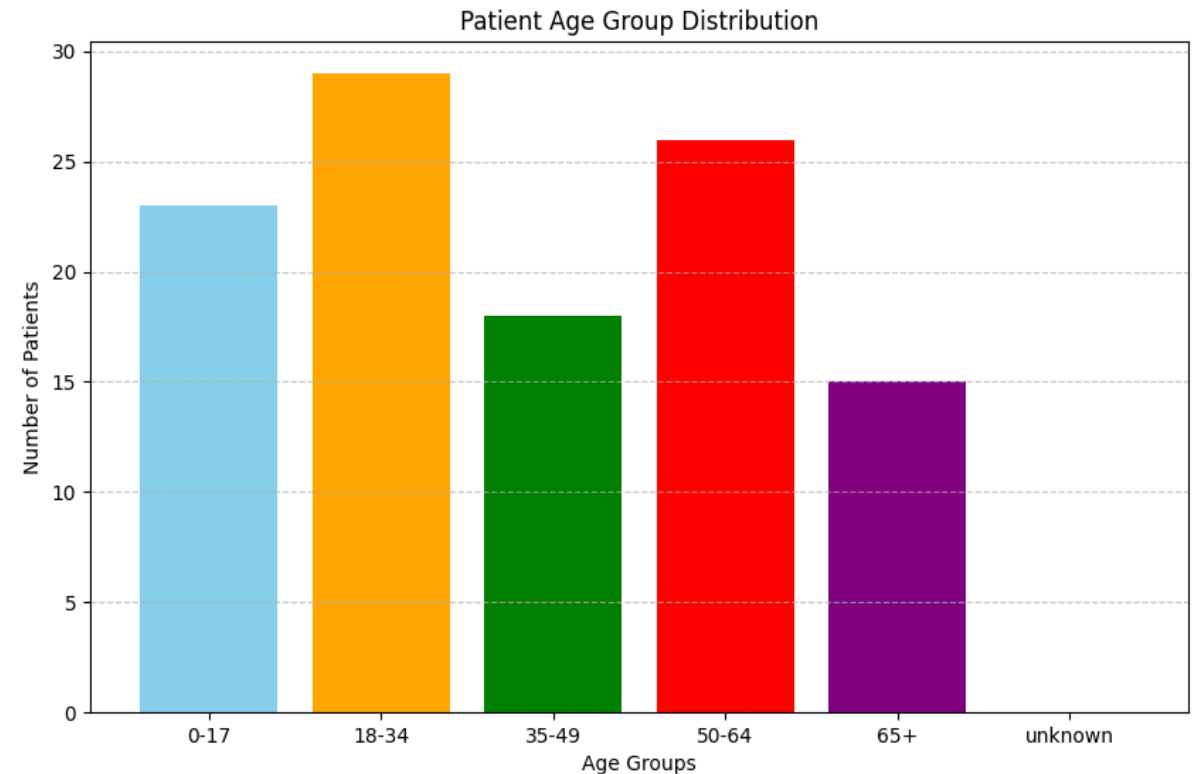
BEST PRACTICES FOR ETL PIPELINES

- **Familiarity with Source APIs:** Understand the structure of source APIs (e.g., OpenEMR, Primary Care EHR) to design the ETL pipeline effectively.
- **Test API Endpoints:** Ensure reliability by testing API endpoints before integrating them into the pipeline.
- **Efficient Data Mapping:** Map the data to align with FHIR hierarchies for consistency and accuracy.
- **Automate Workflows:** Leverage Python (requests, pandas) to automate data processing, enhancing workflow efficiency.



This bar chart illustrates the distribution of patients across various age groups, with the 18–34 group having the largest count (approximately 30) and the "unknown" category having the fewest among 111 patients. The data is derived from OpenEMR FHIR API.

The 35–49 and 65+ groups show smaller numbers compared to the other age groups, highlighting a diverse age demographic.

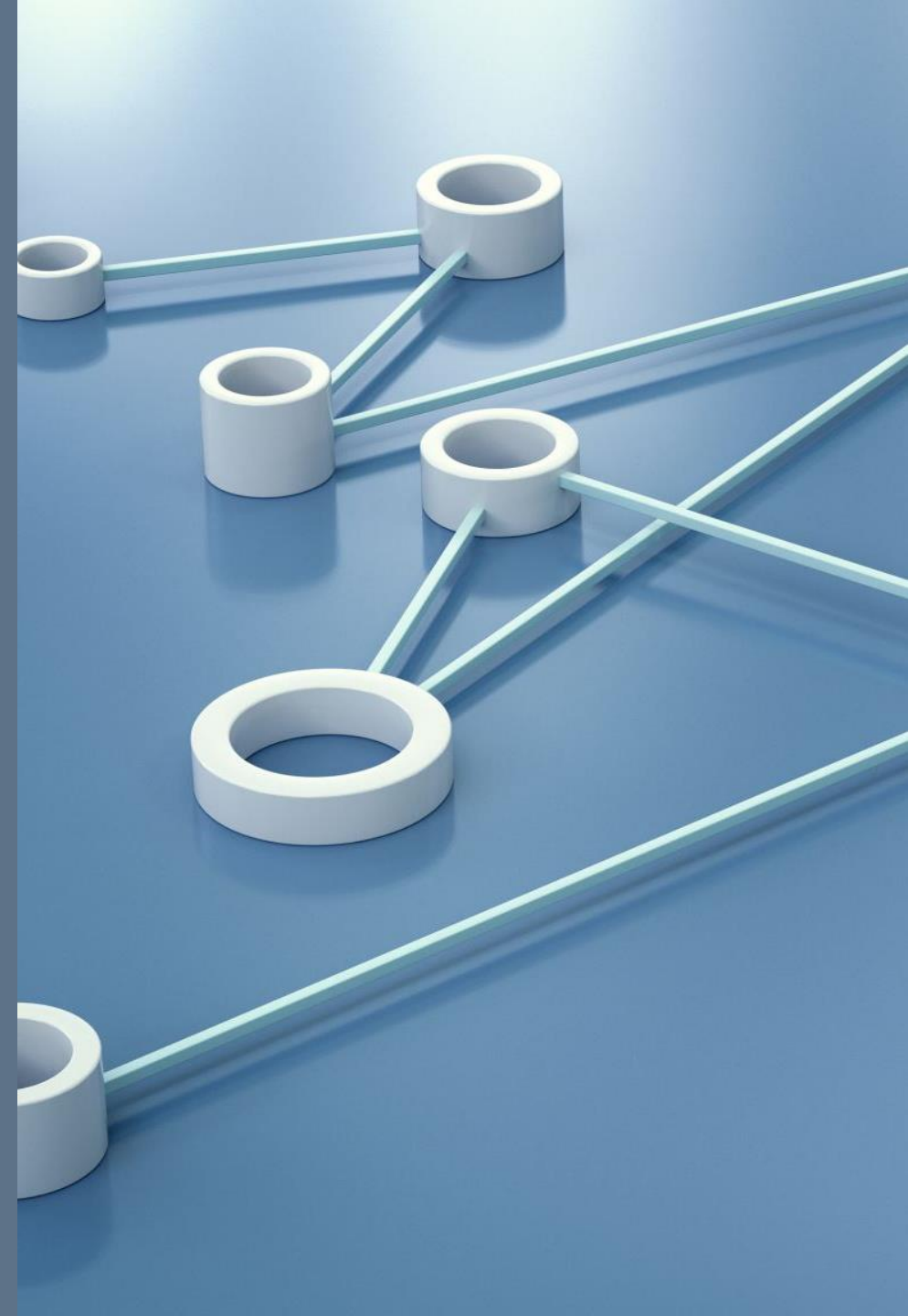


VISUALIZATIONS

CLOSING

The ETL pipeline we developed is valuable for healthcare organizations, as it enables seamless data integration between diverse systems like OpenEMR and the Primary Care EHR. By automating the extraction, transformation, and loading of clinical data, it ensures consistency, enriches data quality through terminological mappings, and enhances interoperability.

We're happy to answer any questions about the challenges, technical details, or impact of this project



THE END